NTLM bridge sidecar for Red Hat Developer Hub

NTLM bridge sidecar for Red Hat Developer Hub

When the corporate forward proxy requires **NTLM** (and will not issue Basic), the Hub process cannot speak NTLM natively. This pattern places a **localhost cntlm sidecar** in the Hub pod: Hub uses `HTTP(S)_PROXY=http://127.0.0.1:3128` with **no proxy credentials**; the sidecar authenticates to the corporate proxy with **NTLM** using a username, password, and domain from a Kubernetes Secret.

Image: Ubuntu-based bridge with **cntlm** (and optional Squid for Kerberos mode in the same image family). Example tag: `registry.example.com/rhdh/ntlm-cntlm-bridge:0.2.0`.

---

1. End-to-end flow



Hop	Auth	Who authenticates
Hub → sidecar (`127.0.0.1:3128`)	**None**	Same pod / localhost only

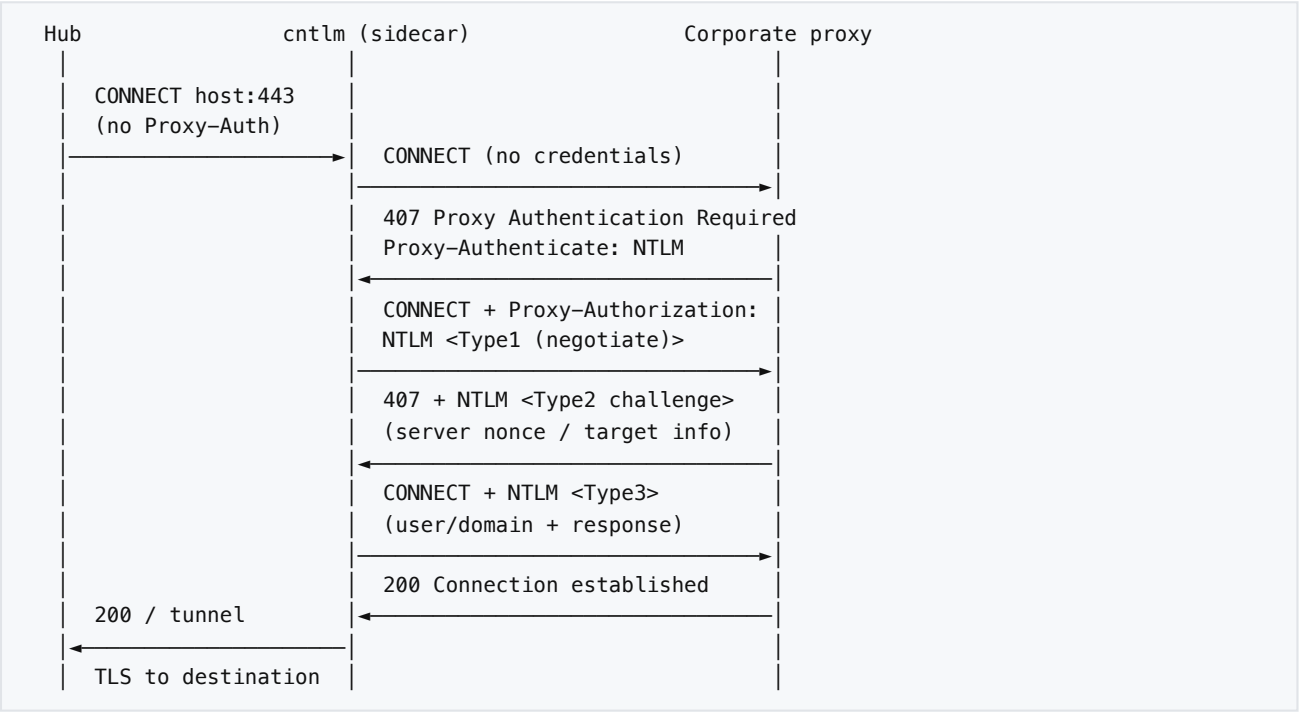
Hop	Auth	Who authenticates
Sidecar → corporate proxy	**NTLM**	Sidecar (**cntlm** + Secret credentials)
Tunnel → SaaS	SaaS credential	Hub plugins, inside TLS after `CONNECT`

Hub never handles NTLM. Plugins continue to use `fetch`; Node's `EnvHttpProxyAgent` only sees `HTTP(S)_PROXY`.

---

2. How cntlm speaks NTLM to the corporate proxy

NTLM over HTTP proxies is a **multi-leg challenge** on the parent hop. cntlm owns that dance so Hub never does.



Step	Wire detail
Challenge	Parent returns **407** with `Proxy-Authenticate: NTLM` (often also `Connection: keep-alive`)
Type 1	cntlm sends an NTLM negotiate message (`Proxy-Authorization: NTLM …`)
Type 2	Parent returns a challenge (nonce; real AD also sends **target info** for NTLMv2)
Type 3	cntlm computes the response from **Username / Password / Domain** and retries
Success	Parent returns **200** on `CONNECT` (or proceeds on plain HTTP GET)

cntlm role: open forward proxy on loopback with **no client auth**; on the parent hop it is the NTLM **client**. Credentials are never placed in Hub's HTTP_PROXY URL.

Auth dialect:

Setting	When
`Auth NTLM` (+ typical `Flags`)	Lab fake helpers and some older parents that omit target-info
`Auth NTLMv2`	Most real Active Directory / enterprise NTLM proxies

Wrong dialect → failures such as "No target info block. Cannot do NTLMv2!" or persistent **407**.

---

3. Why a sidecar (not a separate bridge Deployment)

Sidecar	Separate bridge Service
Hub uses `http://127.0.0.1:3128`	Hub uses `http://bridge.<ns>.svc:3128`
No-auth cntlm is never a cluster Service	No-auth listener must be tightly NetworkPolicy-scoped
NTLM Secret env mounts on the Hub pod	Credentials on a shared volume or separate pod
Shares Hub pod lifecycle	Independent scale / failure domain

Network policy should allow the Hub namespace to reach the corporate proxy VIP.

---

4. Sidecar container specification

Enabled via Hub chart values (`kerberosBridgeSidecar.enabled: true` and `proxyAuthMode: ntlm`) on the Backstage CR `deployment.patch`.

4.1 Identity

Field	Value
Container name	`cntlm` (Negotiate mode uses `kerberos-bridge`)
Image	Bridge image that includes the **cntlm** binary (Ubuntu lab image)
User	Non-root; under OpenShift `restricted-v2` do **not** hard-code a fixed `runAsUser` outside the namespace range

Field	Value
Capabilities	`drop: [ALL]`, `allowPrivilegeEscalation: false`, `seccompProfile: RuntimeDefault`

4.2 Listen port (Hub-facing)

Field	Requirement
`containerPort`	3128 (name `proxy`)
cntlm `Listen`	**`127.0.0.1:3128`** (loopback only). Must match Hub `HTTP_PROXY`.
Client auth on listen	**None**. Safe **only** with the loopback bind.
Probes	**exec** `bash -c 'echo >/dev/tcp/127.0.0.1/3128'` (not `tcpSocket` to the pod IP)

4.3 Credentials (not a keytab)

Field	Requirement
Secret	e.g. `kerberos-bridge-ntlm`
Keys	`username`, `password`, `domain`
Injection	Env `NTLM_USERNAME` / `NTLM_PASSWORD` / `NTLM_DOMAIN` via `secretKeyRef`
Keytab volume	**Not mounted** in NTLM mode

Without username/password the entrypoint exits and cntlm never listens.

4.4 Environment (sidecar)

Env	Purpose	Example
`PROXY_AUTH_MODE`	Must be `ntlm`	`ntlm`
`CORPORATE_PROXY_HOSTNAME`	cntlm `Proxy` host	`proxy.example.com`
`CORPORATE_PROXY_PORT`	Parent listen port	`3128`
`NTLM_USERNAME`	From Secret	`svc-rhdh-proxy`
`NTLM_PASSWORD`	From Secret	*(rotated secret)*
`NTLM_DOMAIN`	Windows / proxy domain	`EXAMPLE`
`BRIDGE_LISTEN_ADDR`	Sidecar default `127.0.0.1`	`127.0.0.1`
`HOME`	Writable home if needed	`/tmp`

Kerberos-only env (`REALM`, `KDC_HOSTNAME`, `KRB5_KTNAME`, …) is unused when `PROXY_AUTH_MODE=ntlm`.

4.5 cntlm behaviour (parent / corporate hop)

```
Username <user>
Password <pass>
Domain <domain>
Proxy <CORP_HOST>:<CORP_PORT>
Listen 127.0.0.1:3128
Gateway yes
Auth NTLM          # or Auth NTLMv2 for real AD
Flags 0x06820000   # tune per parent
```

Directive	Meaning
`Listen 127.0.0.1:3128`	Hub-only; no cluster exposure
`Proxy …`	All Hub traffic is forwarded to the corporate parent
`Gateway yes`	cntlm acts as a gateway/forwarder for clients
`Auth NTLM` / `NTLMv2`	NTLM dialect used on the parent hop

Entrypoint:

1. Require `NTLM_USERNAME` and `NTLM_PASSWORD`.
2. Render cntlm config from the template (escape special characters in the password).
3. `cntlm -f -c /tmp/cntlm.conf` (foreground).

---

5. Requirements to reach the corporate proxy

5.1 NTLM client identity (sidecar)

Requirement	Detail
Account	Domain (or local proxy) account allowed to authenticate **to** the proxy as a client
Password	Stored in a Secret; rotate with IAM policy
Domain	NetBIOS / NTLM domain string the proxy expects
Dialect	NTLMv2 for typical AD; confirm with the proxy team

5.2 Corporate proxy

Requirement	Detail
NTLM enabled	Proxy challenges with `Proxy-Authenticate: NTLM`

Requirement	Detail
Account accepted	Same user/domain the sidecar sends
Allow list	Destinations Hub will `CONNECT` to (SaaS hosts)
Network path	Hub namespace → proxy host:port
Keep-alive	Prefer keep-alive during the Type 1→2→3 exchange

5.3 Environment mapping

Concept	Typical enterprise equivalent
NTLM Secret	Vault/ESO → Kubernetes Secret (`username` / `password` / `domain`)
cntlm sidecar	Custom unsupported bridge in the Hub pod
Corporate proxy	Existing forward proxy with NTLM (no Basic)
Allow list	Proxy policy for SaaS egress
Network path	Firewall / NSG / NetworkPolicy to the proxy VIP

SaaS credentials remain in Hub Secrets as today; they are not used for proxy NTLM.

---

6. Hub container settings

Setting	Value
`HTTP_PROXY`	`http://127.0.0.1:3128` (**no** `user:pass@`)
`HTTPS_PROXY`	Same URL
`NO_PROXY`	In-cluster and registry hosts as required; omit SaaS hosts that must use the proxy
`NODE_OPTIONS`	`--no-node-snapshot --use-env-proxy`
Plugin-install init	Keep off the Hub proxy path

---

7. Example Helm values

```yaml
httpProxy:
  enabled: true
  nodeOptions: "--no-node-snapshot --use-env-proxy"

kerberosBridgeSidecar:
  enabled: true
  proxyAuthMode: ntlm
  ntlmSecret: kerberos-bridge-ntlm
```

```
  image: registry.example.com/rhdh/ntlm-cntlm-bridge:0.2.0
  port: 3128
  corporateProxyHostname: proxy.example.com
  corporateProxyPort: "3128"
```

Example Secret keys: `username`, `password`, `domain`.

---

8. Operational checklist

1. Corporate proxy reachable and challenges with NTLM; account valid.
2. NTLM Secret present in the Hub namespace; image pullable.
3. Hub `HTTP_PROXY` / `HTTPS_PROXY` set to `http://127.0.0.1:3128`.
4. Hub pod includes container `cntlm` with `PROXY_AUTH_MODE=ntlm`; readiness uses the loopback exec probe.
5. Smoke test from the Hub container:

```
oc -n <hub-namespace> exec deploy/<backstage-deploy> -c backstage-backend -- \
  sh -c 'echo "$HTTPS_PROXY"; node --use-env-proxy -e
        "fetch(\"https://example.com\").then(r=>console.log(r.status))"'
```

Expect `http://127.0.0.1:3128` and HTTP **200** for an allow-listed host.

Failure modes

Symptom	Likely cause
Sidecar not Ready; logs missing username/password	Secret not mounted / wrong keys
Sidecar Ready but Hub gets **407** via proxy	Wrong user/domain/password; account denied on proxy
Logs: `Cannot do NTLMv2` / no target info	Parent lacks NTLMv2 target-info — try `Auth NTLM`, or fix parent for NTLMv2
Persistent CONNECT failures	Allow-list deny, wrong parent host/port, firewall
Hub `ENOTFOUND bridge.…`	Secret still points at a Service URL — use localhost
Open proxy risk	cntlm must stay on `127.0.0.1` only

---

9. Related chart paths

Path	Role
`charts/kerberos-lab/images/bridge/`	Ubuntu Squid + **cntlm** image sources
`charts/kerberos-lab/SIDECAR-NTLM.md`	Lab-oriented NTLM sidecar detail

Path	Role
`charts/developer-hub/templates/backstage.yaml`	Sidecar on the Backstage CR (`proxyAuthMode: ntlm`)
`charts/developer-hub/values.yaml`	`kerberosBridgeSidecar.*` / `ntlmSecret` knobs